

Building Neural Networks with PyTorch `nn.Module`

Registered Layers, Built-in Losses, and Optimizers

Abstract

PyTorch's `torch.nn` package turns manually managed tensor computations into structured neural-network modules. Layers assigned to a module become registered components; their weights and biases are discoverable, differentiable, serializable, and directly consumable by optimizers. This chapter develops a one-neuron classifier, extends it to a hidden-layer network, simplifies it with `nn.Sequential`, and then upgrades a manual training pipeline with `nn.BCELoss` and `torch.optim.SGD`.

Contents

1	From a manual pipeline to framework-managed components	1
2	What <code>torch.nn</code> provides	2
3	The <code>nn.Module</code> contract	3
4	Calling and inspecting a one-neuron model	4
5	Adding a hidden layer	5
6	Simplifying composition with <code>nn.Sequential</code>	5
7	Improvement version 1: registered model, manual loss and update	6
8	Improvement version 2: built-in binary cross-entropy	8
9	Improvement version 3: optimizer-managed updates	8
10	Evaluation and train/evaluation state	9
11	Parameters, module trees, and <code>state_dict</code>	10
12	Complete conceptual comparison	11
13	Key takeaways	11
14	Review questions	11

1 From a manual pipeline to framework-managed components

A manual binary classifier can be mathematically correct while still demanding repetitive bookkeeping: create weight and bias tensors, enable gradients, write an activation, implement a loss, mutate every

parameter under `no_grad`, and clear every gradient separately. Four replacements preserve the same learning process while reducing those risks:

1. describe the architecture with `nn.Module` and built-in layers;
2. use built-in activation modules such as `nn.Sigmoid` or `nn.ReLU`;
3. use a tested loss module such as `nn.BCELoss`;
4. delegate updates and gradient clearing to an optimizer from `torch.optim`.

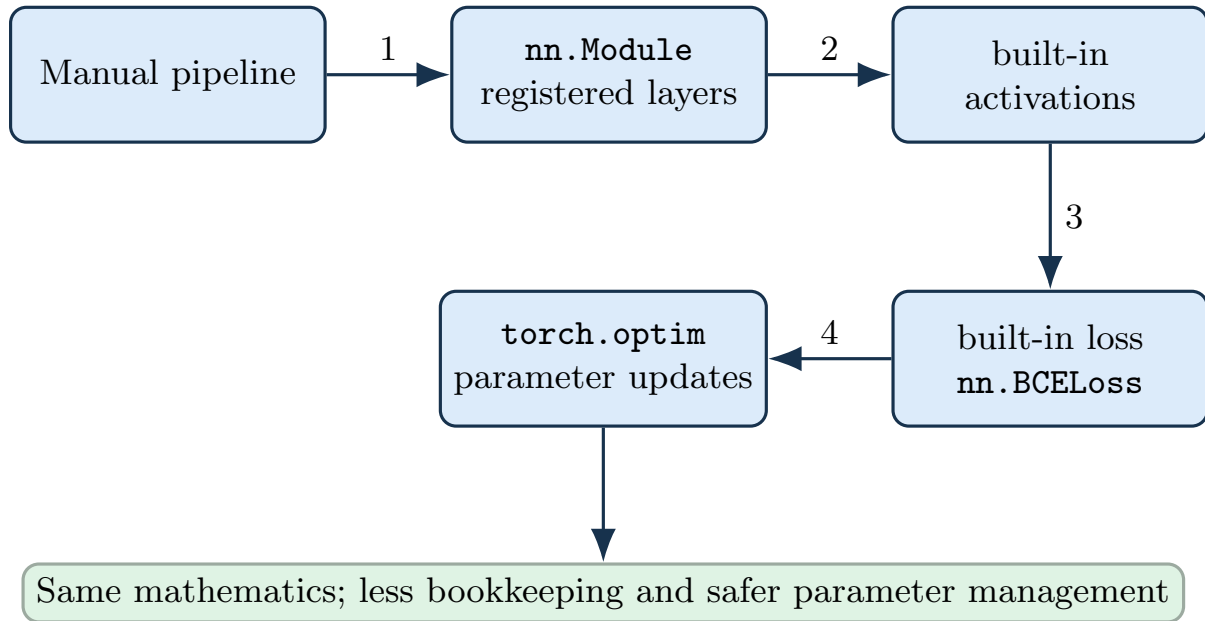


Figure 1: Four framework abstractions replace manual bookkeeping without changing the underlying forward–loss–backward–update cycle. [Technical reference](#).

2 What `torch.nn` provides

Definition

`torch.nn` is PyTorch’s neural-network library. It supplies trainable layers, activation functions, loss functions, module containers, regularization components, and utilities that cooperate with autograd.

Component	Examples	Purpose
Layers	<code>Linear</code> , convolution, recurrent/LSTM layers	Hold trainable transformations and register parameters.
Activations	<code>ReLU</code> , <code>Sigmoid</code> , <code>Tanh</code>	Introduce nonlinear mappings between layers.
Losses	binary cross-entropy, cross-entropy, regression losses	Turn predictions and targets into a scalar objective.
Containers	<code>Sequential</code> , <code>ModuleList</code>	Organize many submodules without losing registration.
Regularization	dropout and other helpers	Control generalization and support architecture design.

Table 1: Major categories in `torch.nn`.

The abstraction removes implementation detail, not the need to reason about shapes, transformations, parameter counts, loss semantics, or optimization order.

3 The `nn.Module` contract

A custom model normally inherits from `nn.Module`. Its constructor builds persistent components, while `forward` defines how an input moves through them.

```
import torch
import torch.nn as nn

class Model(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.linear = nn.Linear(num_features, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, features):
        out = self.linear(features)
        out = self.sigmoid(out)
        return out
```

Listing 1: Single-neuron binary classifier as a custom module

Inheritance. `class Model(nn.Module)` gives the child model registration, device movement, serialization, hook handling, train/evaluation state, and parameter traversal.

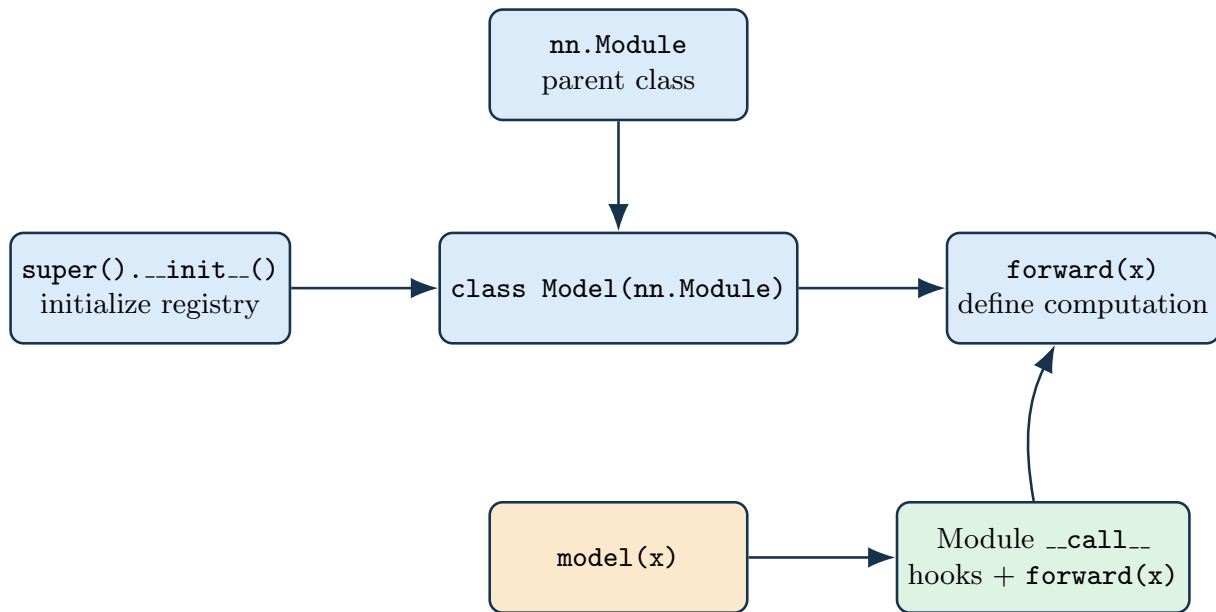
Parent initialization. `super().__init__()` initializes `nn.Module`’s internal registries. Omitting it can prevent submodules and parameters from being managed correctly.

Layer assignment. Assigning `nn.Linear` to `self.linear` registers both the layer and its trainable tensors. There is no need to create raw weights or write `requires_grad=True`; `nn.Linear` creates trainable `Parameter` objects automatically.

Forward definition. The input first undergoes an affine map and then a sigmoid:

$$z = XW^T + b, \quad \hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Here $X \in \mathbb{R}^{B \times d}$ is a batch of B examples with d features, $W \in \mathbb{R}^{1 \times d}$ is the layer’s weight matrix, $b \in \mathbb{R}$ is its bias, $z \in \mathbb{R}^{B \times 1}$ contains logits, and $\hat{y} \in (0, 1)^{B \times 1}$ contains probabilities. The affine transformation learns a decision boundary; sigmoid converts each logit into a binary-class probability.



Prefer invocation over calling **forward** directly

Figure 2: A custom module combines inheritance, parent initialization, a forward definition, and framework-controlled invocation. [Technical reference](#).

4 Calling and inspecting a one-neuron model

The first demonstration uses ten synthetic rows and five features. Random values make exact probabilities non-deterministic, but all shapes and parameter counts are fixed.

```

features = torch.rand(10, 5)
model = Model(features.shape[1])

y_pred = model(features)      # preferred framework invocation
print(y_pred.shape)           # torch.Size([10, 1])
print(model.linear.weight)     # five weights, shape [1, 5]
print(model.linear.bias)       # one bias, shape [1]
print(sum(p.numel() for p in model.parameters())) # 6

# Optional architecture summary used in the demonstration:
# !pip install torchinfo
# from torchinfo import summary
# summary(model, input_size=(10, 5))

```

Listing 2: Create fake data, invoke the model, and inspect its state

Using `features.shape[1]` is more robust than hard-coding 5: the constructor adapts if the table’s feature count changes. Each of ten rows yields one probability, so the output has shape `[10, 1]`.

Call `model(features)` rather than `model.forward(features)`. Module invocation passes through `nn.Module`’s `__call__` machinery before dispatching to `forward`; this preserves hooks and other framework behavior. A direct `forward` call may compute the same tensor in a simple case but bypasses that machinery.

For $d = 5$, parameter count is

$$N_{\text{params}} = 1 \times d + 1 = 5 + 1 = 6.$$

The five coefficients measure the contribution of each feature to the logit; the bias shifts the decision boundary independently of feature values.

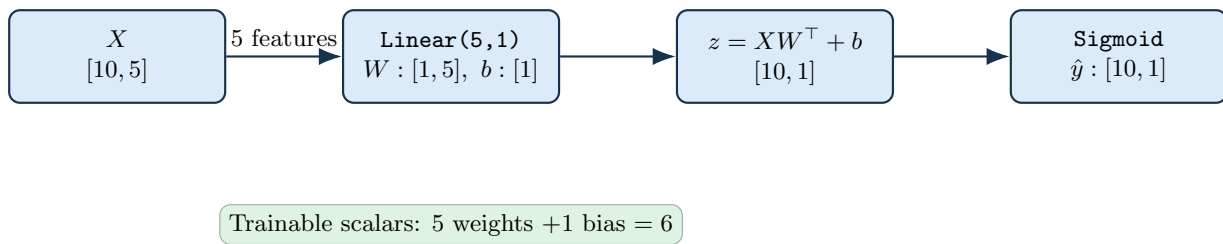


Figure 3: A five-feature affine layer produces one logit and one sigmoid probability per row. [Technical reference](#).

5 Adding a hidden layer

A more expressive network maps five inputs to three hidden units and then to one output. ReLU is used in the hidden layer; sigmoid remains at the binary output.

```

class HiddenModel(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.linear1 = nn.Linear(num_features, 3)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(3, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, features):
        out = self.linear1(features)
        out = self.relu(out)
        out = self.linear2(out)
        out = self.sigmoid(out)
        return out

model = HiddenModel(5)
print(model.linear1.weight.shape) # torch.Size([3, 5])
print(model.linear2.weight.shape) # torch.Size([1, 3])
print(sum(p.numel() for p in model.parameters())) # 22

```

Listing 3: Two-layer network with an explicit forward path

For batch size B , the shape flow is

$$[B, 5] \longrightarrow [B, 3] \longrightarrow [B, 3] \longrightarrow [B, 1] \longrightarrow [B, 1].$$

ReLU computes $\max(0, z)$ elementwise and therefore does not change shape. Its nonlinearity allows the composition to model boundaries that a single affine map cannot.

The first layer has $3 \times 5 = 15$ weights and 3 biases. The second has $1 \times 3 = 3$ weights and 1 bias:

$$N_{\text{params}} = (15 + 3) + (3 + 1) = 22.$$

Every term is a trainable scalar. Parameter count matters because it measures model capacity and memory demand.

6 Simplifying composition with nn.Sequential

Long chains make an explicit `forward` method repetitive. A sequential container applies its children in insertion order.

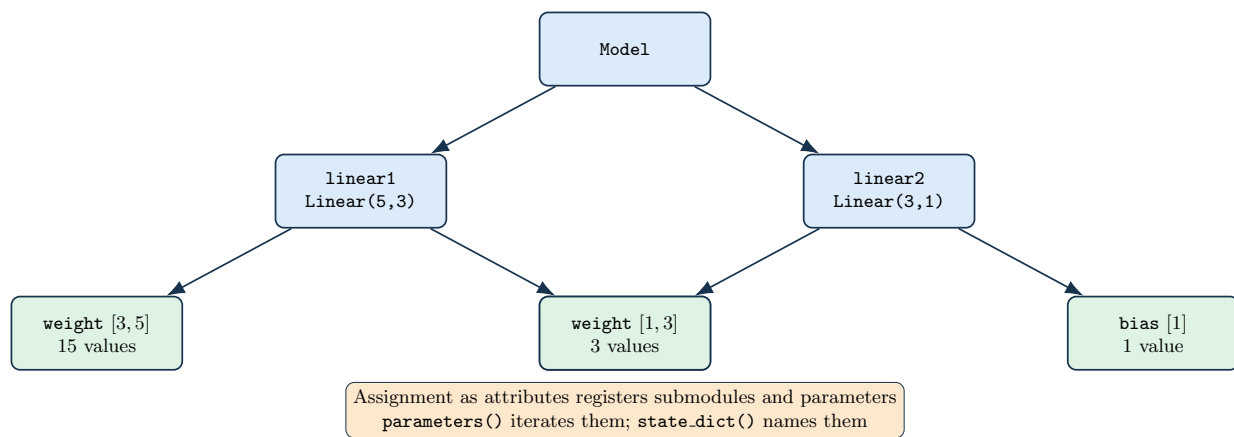


Figure 4: Registered submodules expose a tree of named weights and biases. [Technical reference](#).

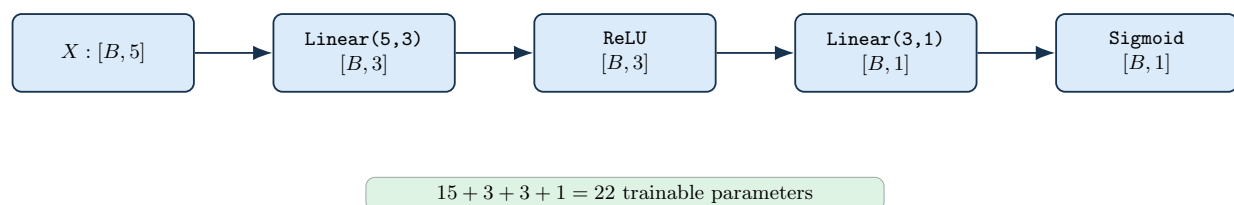


Figure 5: Tensor shapes and parameter arithmetic for the five-to-three-to-one architecture. [Technical reference](#).

```

class SequentialModel(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(num_features, 3),
            nn.ReLU(),
            nn.Linear(3, 1),
            nn.Sigmoid(),
        )

    def forward(self, features):
        return self.network(features)

```

Listing 4: Equivalent hidden network using a sequential container

Each output becomes the next module’s input. The architecture and its 22 trainable values are unchanged; only composition is cleaner.

Pitfall

Arguments inside `nn.Sequential(...)` must be comma-separated. Omitting a comma is a Python syntax error, not a neural-network error. Sequential is appropriate for a simple chain; architectures with branches, skip connections, multiple inputs, or conditional logic usually need an explicit forward method.

7 Improvement version 1: registered model, manual loss and update

The prior breast-cancer example contains 30 input features and one binary output. The first upgrade replaces raw parameter tensors and manual sigmoid code while deliberately retaining the hand-written binary cross-entropy and update logic.

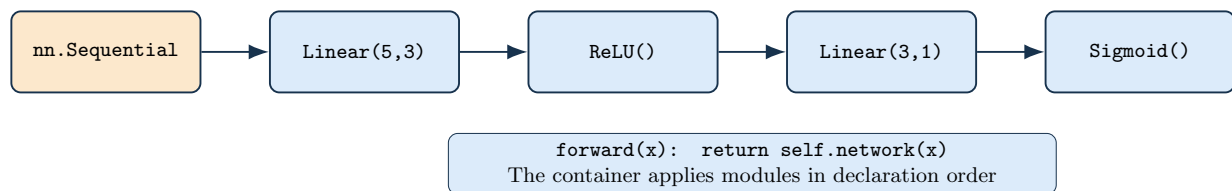


Figure 6: A sequential container owns an ordered chain and reduces the forward method to one call. [Technical reference](#).

```

class BreastCancerModel(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.linear = nn.Linear(num_features, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, features):
        return self.sigmoid(self.linear(features))

    def loss_function(self, y_pred, y_true):
        epsilon = 1e-7
        y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)
        losses = -(
            y_true * torch.log(y_pred)
            + (1 - y_true) * torch.log(1 - y_pred)
        )
        return losses.mean()
  
```

Listing 5: Registered 30-feature classifier with a manual loss

```

learning_rate = 0.1
model = BreastCancerModel(X_train_tensor.shape[1])

for epoch in range(epochs):
    y_pred = model(X_train_tensor)
    loss = model.loss_function(y_pred, y_train_tensor)
    loss.backward()

    with torch.no_grad():
        model.linear.weight -= learning_rate * model.linear.weight.grad
        model.linear.bias -= learning_rate * model.linear.bias.grad

    model.linear.weight.grad.zero_()
    model.linear.bias.grad.zero_()
  
```

Listing 6: Training version 1 with explicit parameter mutation

The registered layer creates a weight matrix of shape `[1, 30]` and a bias of shape `[1]`: 31 trainable scalars. Autograd tracks the affine and sigmoid operations automatically and fills `weight.grad` and `bias.grad` after `backward()`.

Pitfall

The attribute is `model.linear.weight`, singular. Using `weights` raises an attribute error. Registration makes parameters discoverable, but it does not invent alternate attribute names.

The intermediate reconstruction produced accuracy around 66%, but this number is not deterministic because initialization and potentially data handling vary. Its purpose is to verify that the registered model participates in the same training pipeline.

8 Improvement version 2: built-in binary cross-entropy

For probabilities already passed through sigmoid, `nn.BCELoss()` computes

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

Here B is batch size, $y_i \in \{0,1\}$ is the target, and $\hat{y}_i \in (0,1)$ is the predicted probability. The expression penalizes confident wrong probabilities most strongly and returns one mean scalar suitable for backpropagation.

```

criterion = nn.BCELoss()

y_pred = model(X_train_tensor)           # shape [455, 1]
y_target = y_train_tensor.view(-1, 1)    # shape [455, 1]
loss = criterion(y_pred, y_target)
print(loss.shape)                        # torch.Size([1])

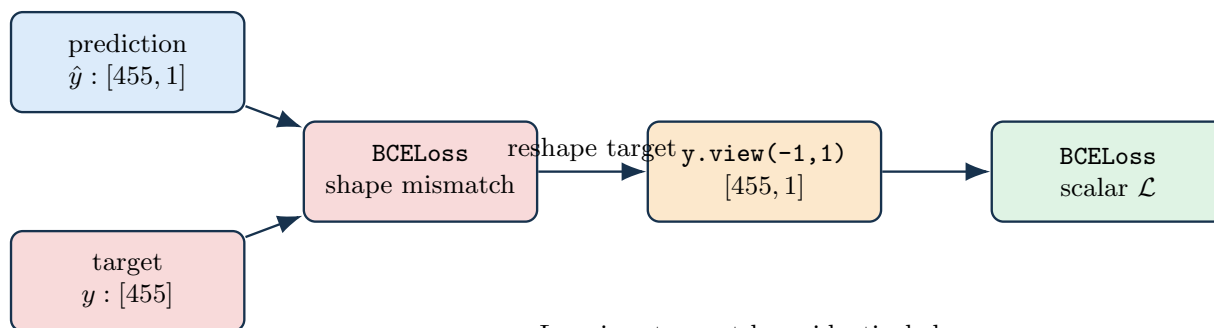
```

Listing 7: Built-in binary cross-entropy with target-shape correction

The demonstrated first attempt supplied prediction shape `[455, 1]` and target shape `[455]`, which `BCELoss` rejected. Equal element counts do not imply equal tensor shapes. `view(-1, 1)` infers the row count and introduces the required singleton output dimension. It is a reshape-style view when the storage layout is compatible; `reshape(-1, 1)` is another clear option.

Intuition

One prediction must line up with one target at the same tensor coordinate. Matching shapes makes this correspondence explicit and prevents accidental broadcasting or ambiguous loss semantics.



Loss inputs must have identical shape

Figure 7: Binary cross-entropy requires predictions and targets to have the same shape. [Technical reference](#).

A numerically stronger modern alternative is to remove the final sigmoid and use `nn.BCEWithLogitsLoss`, which combines both steps stably. The demonstrated pipeline uses `Sigmoid` plus `BCELoss`, so that arrangement is preserved here.

9 Improvement version 3: optimizer-managed updates

`torch.optim` provides parameter-update algorithms including SGD, momentum variants, RMSprop, Adam, and Adadelta. Optimizers can also cooperate with learning-rate schedulers and apply options such as weight decay.

An optimizer must be created *after* its model because it receives an iterator over that model's registered parameters.

```

learning_rate = 0.1
model = BreastCancerModel(X_train_tensor.shape[1])
criterion = nn.BCELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
y_target = y_train_tensor.view(-1, 1)

for epoch in range(epochs):
    optimizer.zero_grad()           # clear old gradient buffers
    y_pred = model(X_train_tensor)  # forward pass
    loss = criterion(y_pred, y_target)
    loss.backward()                 # compute current gradients
    optimizer.step()                # update every managed parameter

```

Listing 8: Clean training pipeline using BCELoss and SGD

`model.parameters()` returns an iterator over all registered trainable parameters, including tensors nested inside child modules. This removes the need to name each weight and bias in the training loop.

For plain SGD, each parameter tensor θ is updated as

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_t.$$

The symbol t denotes the update index, η is the learning rate, and $\nabla_{\theta} \mathcal{L}_t$ is the current loss derivative with respect to θ . `optimizer.step()` applies this rule to all managed tensors. `optimizer.zero_grad()` clears their stored derivatives so that the next backward pass starts clean.

Clearing before `backward()` gives an easy-to-audit order: clear, compute, differentiate, update. Clearing after `step()` can also work if done exactly once per iteration, but omission causes accumulation.

Pitfall

Defining `torch.optim.SGD(model.parameters(), ...)` before `model` exists raises a name error. Creating parameters after the optimizer is also unsafe because newly created tensors are not automatically added to the optimizer's existing parameter groups.

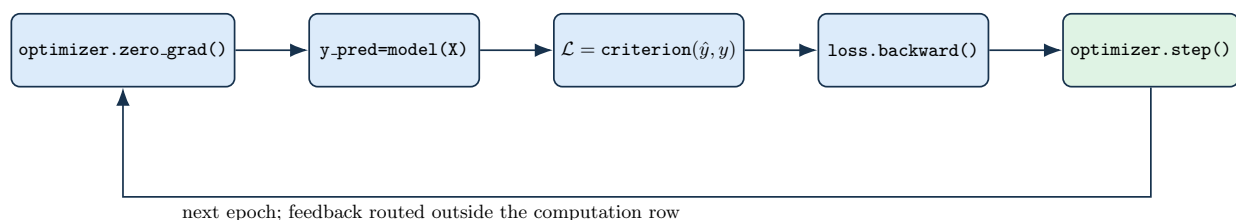


Figure 8: One optimizer-managed epoch, with the next-epoch feedback path routed outside the main computation row. [Technical reference](#).

10 Evaluation and train/evaluation state

Evaluation should not create gradients or update parameters. A probability threshold converts outputs to classes.

```

model.eval()
with torch.no_grad():
    y_prob = model(X_test_tensor)
    y_pred = (y_prob >= 0.5).to(y_test_tensor.dtype)
    y_true = y_test_tensor.view(-1, 1)
    accuracy = (y_pred == y_true).float().mean()

print(y_prob.shape)                # typically [114, 1]
print(f"Accuracy: {100 * accuracy.item():.2f}%")
model.train()

```

Listing 9: Evaluate the trained binary classifier

The threshold rule is $\hat{c}_i = \mathbf{1}[\hat{y}_i \geq 0.5]$, where $\hat{c}_i$ is the predicted class. Accuracy is the number of equal predicted and true labels divided by the test-set size. `eval()` changes the state of modules such as dropout and batch normalization; it does not itself disable gradients. `torch.no_grad()` handles gradient recording. This architecture has neither dropout nor batch normalization, but using both mechanisms establishes the correct general pattern.

11 Parameters, module trees, and state_dict

Registration connects model definition to optimization and persistence. The following inspection is a useful extension of the demonstrated weight, bias, and summary checks.

```

for name, module in model.named_modules():
    print("module:", name, type(module).__name__)

for name, parameter in model.named_parameters():
    print(name, tuple(parameter.shape), parameter.requires_grad)

for key, tensor in model.state_dict().items():
    print(key, tuple(tensor.shape))

# Expected keys for BreastCancerModel:
# linear.weight (1, 30)
# linear.bias (1,)

```

Listing 10: Inspect registered modules, parameters, and serializable state

`named_modules()` traverses the architecture. `named_parameters()` yields registered trainable tensors. `state_dict()` maps stable names to parameters and persistent buffers, forming the standard payload for saving and loading learned state. Activations such as `Sigmoid` appear as modules but contribute no trainable tensors.

12 Complete conceptual comparison

Concern	Manual form	Framework-managed form
Architecture	Raw weight/bias tensors and explicit tensor operations	Registered <code>nn.Linear</code> and activation modules
Forward invocation	Explicit custom method call	<code>model(X)</code> dispatches to <code>forward</code>
Differentiation	Manually enable <code>requires_grad</code>	Layer parameters require gradients automatically
Loss	Hand-written logarithms and averaging	Callable <code>nn.BCELoss</code> criterion
Update	Per-tensor subtraction under <code>no_grad</code>	<code>optimizer.step()</code>
Gradient reset	Per-tensor <code>grad.zero_()</code>	<code>optimizer.zero_grad()</code>
Inspection	Know each tensor attribute in advance	<code>parameters()</code> , named traversal, summaries
Persistence	Assemble tensors manually	<code>state_dict()</code>

Table 2: Manual mechanics and their idiomatic PyTorch counterparts.

13 Key takeaways

1. `nn.Module` is the base abstraction that organizes computation and state.
2. Always initialize the parent with `super().__init__()` before assigning layers.
3. Submodules assigned as attributes register their weights and biases automatically.
4. Define computation in `forward`, but invoke the model through `model(inputs)`.
5. Shapes remain a programmer responsibility: a $[B, 1]$ prediction requires a compatible $[B, 1]$ binary target.
6. `nn.Sequential` makes straight-line compositions concise; explicit forward logic remains necessary for complex data flow.
7. `model.parameters()` links registered tensors to an optimizer.
8. A clean training iteration is: clear gradients, forward pass, scalar loss, backward pass, optimizer step.
9. `state_dict()` gives registered state stable names for inspection and persistence.

14 Review questions

1. Why can omitting `super().__init__()` break registration?
2. Why is `model(x)` preferred over `model.forward(x)`?
3. How are 22 trainable values obtained for a $5 \rightarrow 3 \rightarrow 1$ network?
4. What does assigning `nn.Linear` to a model attribute accomplish beyond storing a Python object?
5. Why does `BCELoss` reject shapes $[455, 1]$ and $[455]$?
6. What is the difference between `optimizer.zero_grad()` and `optimizer.step()`?
7. Which tensors appear in `state_dict()`, and why does `Sigmoid` add no parameter entries?
8. Why are `eval()` and `no_grad()` separate operations?